

|             |           |             |                             |                 |   |                       |   |   |   |   |
|-------------|-----------|-------------|-----------------------------|-----------------|---|-----------------------|---|---|---|---|
| Course Code | 21CSE356T | Course Name | NATURAL LANGUAGE PROCESSING | Course Category | E | PROFESSIONAL ELECTIVE | L | T | P | C |
|             |           |             |                             |                 |   |                       | 2 | 1 | 0 | 3 |

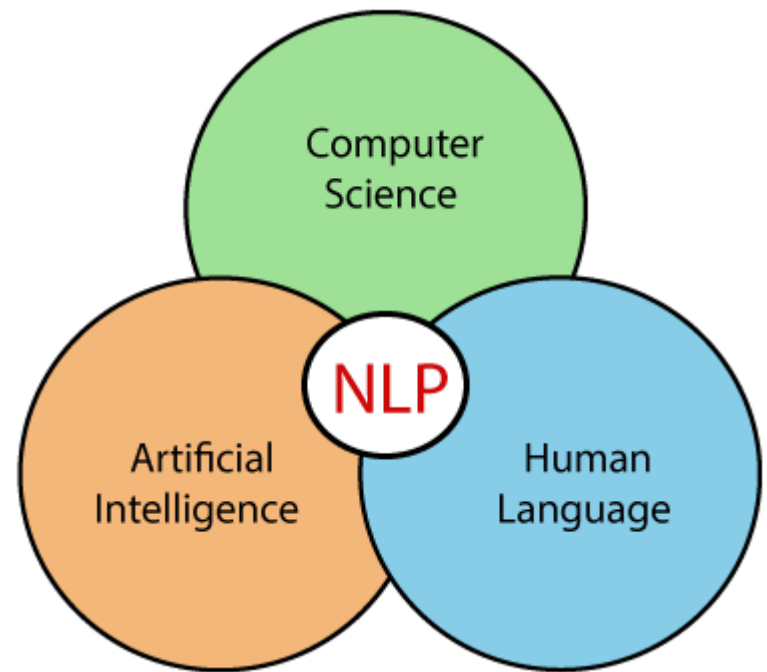
|                            |                     |                               |     |                     |     |
|----------------------------|---------------------|-------------------------------|-----|---------------------|-----|
| Pre-requisite Courses      | Nil                 | Co-requisite Courses          | Nil | Progressive Courses | Nil |
| Course Offering Department | School of Computing | Data Book / Codes / Standards | Nil |                     |     |

| Course Learning Rationale (CLR): |                                                                                                                  | The purpose of learning this course is to:           |   |   |   |   |   |   |   |   |    |    |    | Program Outcomes (PO) |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    | Program Specific Outcomes |       |       |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|---|---|---|---|---|---|---|---|----|----|----|-----------------------|------------------|---------------------------------|--------------------------------------------|-------------------|--------------------------|------------------------------|--------|------------------------|---------------|------------------------|--------------------|---------------------------|-------|-------|
| CLR-1:                           | understand the fundamentals behind the Language processing and perform word level analysis                       | 1                                                    | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | Engineering Knowledge | Problem Analysis | Design/development of solutions | Conduct investigations of complex problems | Modern Tool Usage | The engineer and society | Environment & Sustainability | Ethics | Individual & Team Work | Communication | Project Mgt. & Finance | Life Long Learning | PSO-1                     | PSO-2 | PSO-3 |
| CLR-2:                           | understand the syntactic processing and probabilistic context-free grammars                                      |                                                      |   |   |   |   |   |   |   |   |    |    |    |                       |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CLR-3:                           | conceive the basics of the knowledge representation, inference, and discourse analysis                           |                                                      |   |   |   |   |   |   |   |   |    |    |    |                       |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CLR-4:                           | recognize the significance of transformer-based models                                                           |                                                      |   |   |   |   |   |   |   |   |    |    |    |                       |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CLR-5:                           | understand the natural language processing applications and to learn how to apply basic algorithms in this field |                                                      |   |   |   |   |   |   |   |   |    |    |    |                       |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| Course Outcomes (CO):            |                                                                                                                  | At the end of this course, learners will be able to: |   |   |   |   |   |   |   |   |    |    |    |                       |                  |                                 |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CO-1:                            | exhibit knowledge on text preprocessing techniques and perform word level analysis                               | 3                                                    | 3 | 2 | - | - | - | - | - | - | -  | -  | -  | 2                     | -                | -                               |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CO-2:                            | illustrate approaches to syntax analysis including probabilistic context-free grammars                           | 3                                                    | 3 | 2 | - | - | - | - | - | - | -  | -  | -  | 2                     | -                | -                               |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CO-3:                            | apply approaches to semantics and discourse analysis in NLP                                                      | 3                                                    | 3 | 2 | - | - | - | - | - | - | -  | -  | -  | 2                     | -                | -                               |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CO-4:                            | develop models using transfer learning approaches                                                                | 3                                                    | - | - | 3 | 3 | - | - | - | - | -  | -  | -  | 2                     | -                | -                               |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |
| CO-5:                            | implement applications that use Natural Language Processing approaches                                           | -                                                    | - | 2 | 3 | 3 | - | - | - | - | -  | -  | -  | -                     | -                | -                               |                                            |                   |                          |                              |        |                        |               |                        |                    |                           |       |       |

|                                                                                                                                                                                                                                                                                                                                               |               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>Unit-1 - Overview and Word Level Analysis</b>                                                                                                                                                                                                                                                                                              | <b>9 Hour</b> |
| Introduction to Natural Language Processing, Applications of NLP, Levels of NLP, Regular Expressions, Morphological Analysis, Tokenization, Stemming, Lemmatization, Feature extraction: Term Frequency (TF), Inverse Document Frequency (IDF), Modeling using TF-IDF, Parts of Speech Tagging, Named Entity Recognition, N-grams, Smoothing. |               |
| <b>Unit-2 - Syntax Analysis</b>                                                                                                                                                                                                                                                                                                               | <b>9 Hour</b> |
| Context Free Grammars, Grammar Rules for English, Top-Down Parsing, Bottom-Up Parsing, Ambiguity, CKY Parsing, Dependency Parsing, Earley Parsing - Probabilistic Context-Free Grammars                                                                                                                                                       |               |
| <b>Unit-3 - Semantic and Discourse Analysis</b>                                                                                                                                                                                                                                                                                               | <b>9 Hour</b> |
| Representing Meaning, Lexical Semantics, Word Senses, Relation between Senses, Word Sense Disambiguation, Word Embeddings, Word2Vec, CBOW, Skip-gram and GloVe, Discourse Segmentation, Text Coherence, Discourse Structure, Reference Resolution, Pronominal Anaphora Resolution, Coreference Resolution                                     |               |
| <b>Unit-4 - Language Models</b>                                                                                                                                                                                                                                                                                                               | <b>9 Hour</b> |
| Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Attention mechanism, Transformer Based Models, Self-attention, multi-headed attention, BERT, RoBERTa, Fine Tuning for downstream tasks, Text classification and Text generation.                                                                                              |               |
| <b>Unit-5 - NLP Applications</b>                                                                                                                                                                                                                                                                                                              | <b>9 Hour</b> |
| Introduction to Chatbot Applications, Retrieval based- Conversation based, Information Extraction and its approaches, Information Retrieval, Semantic Search and Evaluation, Question Answering, Summarization, Extractive Vs Abstractive Summarization, Machine Translation.                                                                 |               |

# Introduction to NLP

- NLP stands for **Natural Language Processing**, which is a part of **Computer Science**, **Human language**, and **Artificial Intelligence**.
- It is the technology that is used by machines to understand, analyse, manipulate, and interpret human's languages.
- It helps developers to organize knowledge for performing tasks such as **translation**, **automatic summarization**, **Named Entity Recognition (NER)**, **speech recognition**, **relationship extraction**, and **topic segmentation**.



# Introduction

- **Natural Language Processing (NLP)** enables computers to understand and interpret human language.
- While computers excel at processing structured data, such as spreadsheets or databases, natural language in its unstructured form (text, speech, etc.) presents a unique challenge.
- NLP bridges this gap by allowing machines to process and understand human languages, making it an essential tool in modern AI systems.

# Steps

- **Sentence segmentation**
- **Word Tokenization**
- **Lemmatization**
- **Predicting Parts of Speech (POS)**
- **Stop Word Removal**
- **Dependency Parsing**
- **Coreference Resolution**

# Sentence segmentation

Sentence segmentation is the first step in NLP, which involves breaking text into individual sentences. This helps the computer understand the structure of the text.

*San Pedro is a town on the southern part of Ambergris Caye in Belize. According to estimates, it has a population of 16,444.*

*San Pedro is a town on the southern part of Ambergris Caye in Belize.*

*According to estimates, it has a population of 16,444.*

# Word Tokenization

Word tokenization divides sentences into smaller components called tokens (words or punctuation). These tokens are crucial for understanding how a sentence is structured.

## Example Input:

San Pedro is a town in Belize.



**Output: Tokens:** ['San', 'Pedro', 'is', 'a', 'town', 'in', 'Belize']

# Predicting Parts of Speech (POS)

POS tagging involves identifying the function of each word in a sentence, such as whether it's a noun, verb, or adjective. This helps determine the role each word plays in the context of the sentence.

**Example Input:** San Pedro is a town.



**Output:** 'San Pedro' – Noun, 'is' – Verb, 'a' – Article, 'town' – Noun

# Lemmatization

Lemmatization converts words to their root forms. For example, “*Buffalo*” and “*Buffaloes*” are both lemmatized to “*Buffalo*,” ensuring that variations of the same word are treated identically.

**Example Input:** There are Buffaloes grazing in the field.



**Output:** Buffalo (root word)



# Stop Word Removal

Stop words (e.g., “a,” “the,” “and”) are common words that provide minimal meaning. Removing them helps reduce noise and improve the efficiency of NLP models.

# Dependency Parsing

- Dependency parsing identifies relationships between words, creating a syntactic tree. This helps understand the grammatical structure of a sentence and the roles of each word.
- **Example Input:** San Pedro is an island in Belize.
- **Output:** Parse Tree: 'San Pedro' (subject) → 'is' (verb) → 'island' (object)
- Noun phrases group related words to represent a specific concept. In the sentence "*The second-largest town in the Belize District,*" we can extract the noun phrase "*second-largest town.*"
-

# Named Entity Recognition (NER)

- NER identifies and categorizes entities such as people, places, or dates in text.
- **Example Input:** San Pedro is a town on the southern part of the island of Ambergris Caye in the Belize District of the nation of Belize, in Central America.
- **Output:**
  - San Pedro: Geographic Entity
  - Ambergris Caye: Geographic Entity
  - Belize: Geographic Entity
  - Central America: Geographic Entity

# Coreference Resolution

- Coreference resolution identifies when two or more expressions in a text refer to the same entity. For example, the word “*it*” might refer to a specific person or thing earlier in the sentence.
- **Example Input:** San Pedro is a town on the southern part of the island of Ambergris Caye. According to 2015 mid-year estimates, the town has a population of about 16,444. It is the second-largest town in the Belize District.
- **Output:** “It” refers to “San Pedro.”

# Techniques Used in NLP

- **Rule-based Methods:** These involve manually created rules and heuristics to process language data. For example, defining patterns in language to extract meaning.
- **Machine Learning (ML) and Deep Learning (DL):** These involve using algorithms to automatically learn from data and improve over time. ML models such as decision trees, support vector machines, and deep learning models like recurrent neural networks (RNNs) and transformers are commonly used in modern NLP.

# Application of NLP

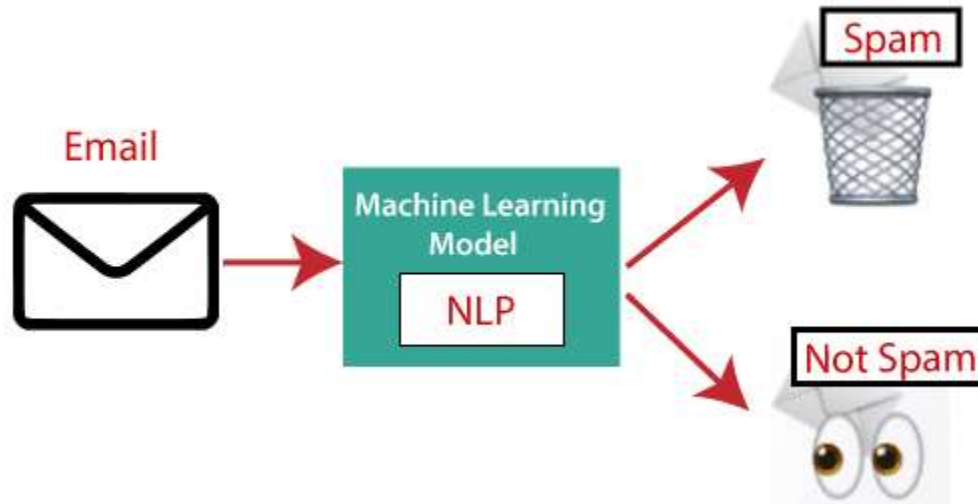
## **Question Answering**

Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language.



# Spam Detection

- Spam detection is used to detect unwanted e-mails getting to a user's inbox.



# Sentiment Analysis

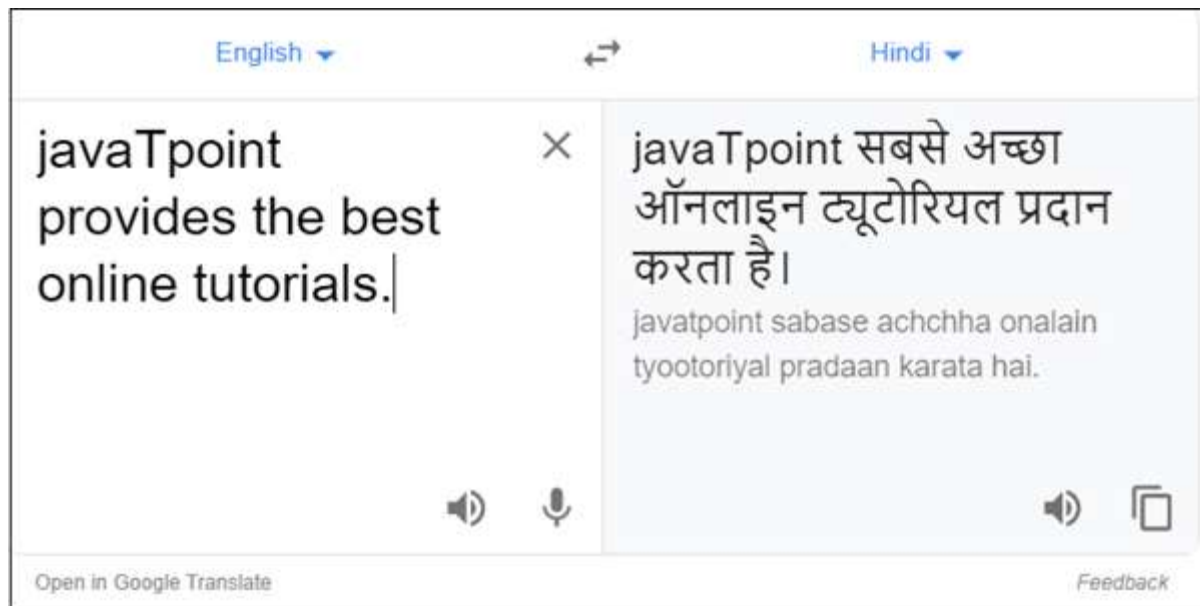
Sentiment Analysis is also known as **opinion mining**. It is used on the web to analyse the attitude, behaviour, and emotional state of the sender. This application is implemented through a combination of NLP (Natural Language Processing) and statistics by assigning the values to the text (positive, negative, or neutral), identify the mood of the context (happy, sad, angry, etc.)





# Machine Translation

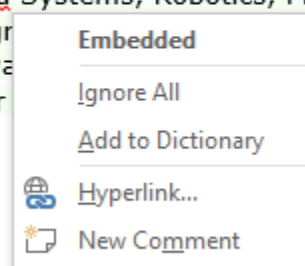
- Machine translation is used to translate text or speech from one natural language to another natural language.
- **Example:** Google Translator



# Spelling correction

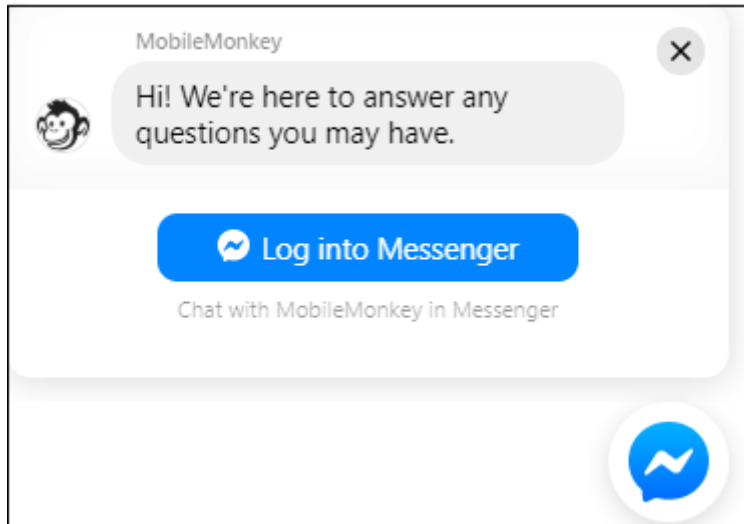
- Microsoft Corporation provides word processor software like MS-word, PowerPoint for the spelling correction.

JavaTpoint offers **Corporate Training, Summer Training, Online Training** and **Winter Training** on Java, Blockchain, Machine Learning, Meanstack, Artificial Intelligence, Kotlin, Cloud Computing, Angular, React, IOT, DevOps, RPA, Virtual Reality, Embedded Systems, Robotics, PHP, .Net, Big Data and Hadoop, Spark, Data Analytics, R Program, Python, Oracle, Web Designing, Spring, Hibernate, Software Development, QTP, Linux, CCNA, C++ and many more technologies. For more information visit [www.javatpoint.com](http://www.javatpoint.com)



# Chatbot

- Implementing the Chatbot is one of the important applications of NLP. It is used by many companies to provide the customer's chat services.



# Applications

- **Information extraction**

Information extraction is one of the most important applications of NLP. It is used for extracting structured information from unstructured or semi-structured machine-readable documents.

## **Natural Language Understanding (NLU)**

- It converts a large set of text into more formal representations such as first-order logic structures that are easier for the computer programs to manipulate notations of the natural language processing.

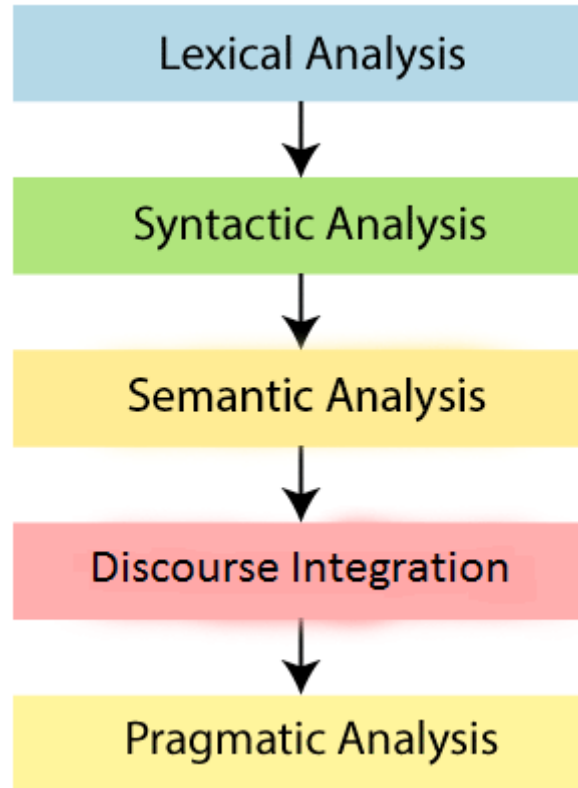
# Speech Recognition

- 
- Speech recognition is used for converting spoken words into text. It is used in applications, such as mobile, home automation, video recovery, dictating to Microsoft Word, voice biometrics, voice user interface, and so on.

# Python NLP tools

- NLTK (Natural Language Toolkit)
- spaCy
- Transformers
- TextBlob
- Gensim

# Levels of NLP



# Lexical Analysis and Morphological

- The first phase of NLP is the Lexical Analysis. This phase scans the source code as a stream of characters and converts it into meaningful lexemes. It divides the whole text into paragraphs, sentences, and words.



# Syntactic analysis

- Syntax describes how a language's words and phrases arrange to form sentences. Syntactic analysis checks word arrangements for proper grammar.
- For instance, the sentence "Dave wrote the paper" passes a syntactic analysis check because it's grammatically correct.
- Conversely, a syntactic analysis categorizes a sentence like "Dave do jumps" as syntactically incorrect.

# Semantic analysis

- Semantics describe the meaning of words, phrases, sentences, and paragraphs. Semantic analysis attempts to understand the literal meaning of individual language selections, not syntactic correctness. However, a semantic analysis doesn't check language data before and after a selection to clarify its meaning.
- For instance, "Manhattan calls out to Dave" passes a syntactic analysis because it's a grammatically correct sentence. However, it fails a semantic analysis. Because Manhattan is a place (and can't literally call out to people), the sentence's meaning doesn't make sense.

# Discourse integration

- Discourse describes communication between 2 or more individuals. Discourse integration analyzes prior words and sentences to understand the meaning of ambiguous language.
- For instance, if one sentence reads, “Manhattan speaks to all its people,” and the following sentence reads, “It calls out to Dave,” discourse integration checks the first sentence for context to understand that “It” in the latter sentence refers to Manhattan.

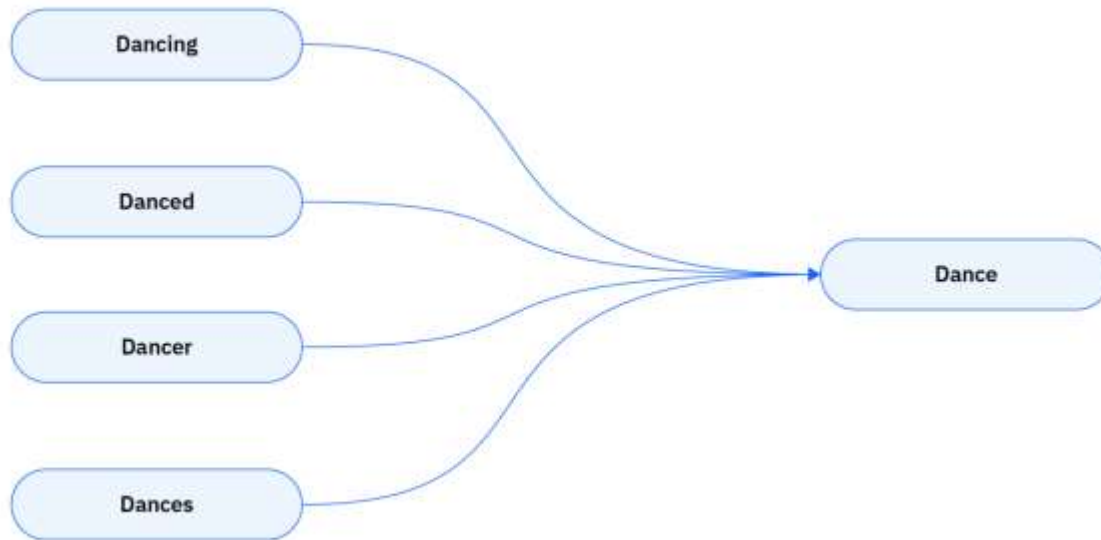
# Pragmatic analysis

- Pragmatism describes the interpretation of language's intended meaning. Pragmatic analysis attempts to derive the intended—not literal—meaning of language.
- For instance, a pragmatic analysis can uncover the intended meaning of “Manhattan speaks to all its people.” Methods like neural networks assess the context to understand that the sentence isn't literal, and most people won't interpret it as such. A pragmatic analysis deduces that this sentence is a metaphor for how people emotionally connect with places.

# Stemming

- **Stemming** is a method in **text processing** that eliminates prefixes and suffixes from words, transforming them into their fundamental or root form,
- The main objective of stemming is to streamline and standardize words, enhancing the effectiveness of the **natural language processing** tasks.
- Simplifying words to their most basic form is called stemming, and it is made easier by stemmers or stemming algorithms

# Stemming example



# Types of Stemmer in NLTK

- **Porter's Stemmer**
- **Lovins Stemmer**
- **Dawson Stemmer**
- **Krovetz Stemmer**
- **Xerox Stemmer**
- **N-Gram Stemmer**

# Porter's Stemmer

- It is one of the most popular stemming methods proposed in 1980.
- It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes.
- This stemmer is known for its speed and simplicity. The main applications of Porter Stemmer include data mining and Information retrieval.



# Porter's Stemmer

- However, its applications are only limited to English words.
- Also, the group of stems is mapped on to the same stem and the output stem is not necessarily a meaningful word.
- The algorithms are fairly lengthy in nature and are known to be the oldest stemmer.

# Porter's Stemmer

- Original words: ['running', 'jumps', 'happily', 'running', 'happily']  
Stemmed words: ['run', 'jump', 'happili', 'run', 'happili']
- **Advantage:** It produces the best output as compared to other stemmers and it has less error rate.
- **Limitation:** Morphological variants produced are not always real words.

# Lovins Stemmer

- It is proposed by Lovins in 1968, that removes the longest suffix from a word then the word is recorded to convert this stem into valid words.

**Example:** sitting -> sitt -> sit

- **Advantage:** It is fast and handles irregular plurals like 'teeth' and 'tooth' etc.
- **Limitation:** It is time consuming and frequently fails to form words from stem.

# Dawson Stemmer

- It is an extension of Lovins stemmer in which suffixes are stored in the reversed order indexed by their length and last letter.
- **Advantage:** It is fast in execution and covers more suffices.
- **Limitation:** It is very complex to implement.

# Krovetz Stemmer

- It was proposed in 1993 by Robert Krovetz. Following are the steps:
  - 1) Convert the plural form of a word to its singular form.
  - 2) Convert the past tense of a word to its present tense and remove the suffix 'ing'.

**Example:** 'children' -> 'child'
- **Advantage:** It is light in nature and can be used as pre-stemmer for other stemmers.
- **Limitation:** It is inefficient in case of large documents.

# Xerox Stemmer

- Capable of processing extensive datasets and generating valid words, it has a tendency to over-stem, primarily due to its reliance on lexicons, making it language-dependent.
- This constraint implies that its effectiveness is limited to specific languages.
- **Example:**
- ‘children’ -> ‘child’
- ‘understood’ -> ‘understand’
- ‘whom’ -> ‘who’
- ‘best’ -> ‘good’

# N-Gram Stemmer

- The algorithm, aptly named n-grams (typically  $n=2$  or  $3$ ), involves breaking words into segments of length  $n$  and then applying statistical analysis to identify patterns. An n-gram is a set of  $n$  consecutive characters extracted from a word in which similar words will have a high proportion of n-grams in common.

**Example:** 'INTRODUCTIONS' for  $n=2$  becomes : \*I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S\*

- **Advantage:** It is based on string comparisons and it is language dependent.
- **Limitation:** It requires space to create and index the n-grams and it is not time efficient.

# N-Gram Stemmer

- **Example:** 'INTRODUCTIONS' for  $n=2$  becomes : \*I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S\*
- **Advantage:** It is based on string comparisons and it is language dependent.
- **Limitation:** It requires space to create and index the n-grams and it is not time efficient.



# Snowball Stemmer

- The Snowball Stemmer, compared to the Porter Stemmer, is multi-lingual as it can handle non-English words. It supports various languages and is based on the 'Snowball' programming language, known for efficient processing of small strings.
- The Snowball stemmer is way more aggressive than Porter Stemmer and is also referred to as Porter2 Stemmer. Because of the improvements added when compared to the Porter Stemmer, the Snowball stemmer is having greater computational speed.

# Lemmatization

- Lemmatization is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item.
- Lemmatization is similar to stemming but it brings context to the words.
- So, it links words with similar meanings to one word.
- Lemmatization is preferred over Stemming because lemmatization does morphological analysis of the words.

# *Examples of lemmatization:*

- -> *rocks : rock*
- -> *corpora : corpus*
- -> *better : good*

# Lemmatization Techniques

- Lemmatization techniques in natural language processing (NLP) involve methods to identify and transform words into their base or root forms, known as lemmas. These approaches contribute to text normalization, facilitating more accurate language analysis and processing in various NLP applications.
- **Rule Based Lemmatization**
- **Dictionary-Based Lemmatization**
- **Machine Learning-Based Lemmatization**

# Rule Based Lemmatization

- Rule-based lemmatization involves the application of predefined rules to derive the base or root form of a word. Unlike machine learning-based approaches, which learn from data, rule-based lemmatization relies on linguistic rules and patterns.
- **Rule:** For regular verbs ending in “-ed,” remove the “-ed” suffix.
- Example:
- Word: “walked”
- Rule Application: Remove “-ed”
- Result: “walk

# Dictionary-Based Lemmatization

- Dictionary-based lemmatization relies on predefined dictionaries or lookup tables to map words to their corresponding base forms or lemmas.
- Each word is matched against the dictionary entries to find its lemma.
- This method is effective for languages with well-defined rules.

# Dictionary-Based Lemmatization

- Suppose we have a dictionary with lemmatized forms for some words:
- ‘running’ -> ‘run’
- ‘better’ -> ‘good’
- ‘went’ -> ‘go’
- Input : I was running to become a better athlete, and then I went home
- Output : I was run to become a good athlete, and then I go home

# Machine Learning-Based Lemmatization

- Machine learning-based lemmatization leverages computational models to automatically learn the relationships between words and their base forms.
- Unlike rule-based or dictionary-based approaches, machine learning models, such as neural networks or statistical models, are trained on large text datasets to generalize patterns in language.



# Need of feature extraction techniques

- Machine Learning algorithms learn from a pre-defined set of features from the training data to produce output for the test data.
- But the main problem in working with language processing is that machine learning algorithms cannot work on the raw text directly.
- So, we need some feature extraction techniques to convert text into a matrix(or vector) of features.

# Regular expression

- Regular expressions (regex) are an essential tool in **Natural Language Processing (NLP)** for tasks involving text pattern matching, searching, and manipulation.
- They are used to find specific sequences of characters, words, or phrases in a body of text. Regular expressions are particularly helpful for pre-processing, cleaning, extracting, and transforming text data before applying NLP algorithms.

# Components

## Basic Components of Regular Expressions:

### 1. **Literals:** Direct characters or strings.

- Example: `cat` matches the string "cat" exactly.

### 2. **Metacharacters:** Special characters that define patterns.

- `.` (dot): Matches any character except a newline.
- `^`: Anchors the match at the beginning of the string.
- `$`: Anchors the match at the end of the string.
- `[]`: Matches any one of the characters inside the square brackets.
  - Example: `[aeiou]` matches any vowel.
- `|`: Alternation (OR), matches either the left or right side.
  - Example: `cat|dog` matches either "cat" or "dog".

# Components of RE

## 3. Quantifiers: Define how many times an element can appear.

- `*` : Matches 0 or more occurrences.
- `+` : Matches 1 or more occurrences.
- `?` : Matches 0 or 1 occurrence.
- `{n,m}` : Matches between n and m occurrences.
  - Example: `a{2,4}` matches "aa", "aaa", or "aaaa".

## 4. Escape Sequences: Used for special characters that have specific meanings in regex.

- `\d` : Matches any digit (0-9).
- `\w` : Matches any word character (letters, digits, and underscores).
- `\s` : Matches any whitespace character (spaces, tabs, line breaks).
- `\b` : Matches a word boundary.
- `\D` : Matches anything that is not a digit.
- `\W` : Matches anything that is not a word character.



# methods of feature extraction

- Bag-of-Words
- TF-IDF

# Bag of Words:

- The bag of words model is used for text representation and feature extraction in natural language processing and information retrieval tasks.
- It represents a text document as a multiset of its words, disregarding grammar and word order, but keeping the frequency of words.
- This representation is useful for tasks such as text classification, document similarity, and text clustering.

- Bag-of-Words is one of the most fundamental methods to transform tokens into a set of features.
- The BoW model is used in document classification, where each word is used as a feature for training the classifier.
- For example, in a task of review based sentiment analysis, the presence of words like *'fabulous'*, *'excellent'* indicates a positive review, while words like *'annoying'*, *'poor'* point to a negative review

# Steps

- **text-preprocessing**
- **create a vocabulary**
- **create a matrix of features**



# text-preprocessing

- converting the entire text into lower case characters.
- removing all punctuations and unnecessary symbols.

# create a vocabulary

- we have a movie review text. Let's consider 3 of these reviews, which are as follows :
- *good movie*
- *not a good movie*
- *did not like*
- Now, we consider all the unique words from the above set of reviews to create a vocabulary, which is going to be as follows :
- {good, movie, not, a, did, like}

# create a matrix of features

| <b>good</b> | <b>movie</b> | <b>not</b> | <b>a</b> | <b>did</b> | <b>like</b> |
|-------------|--------------|------------|----------|------------|-------------|
| <b>1</b>    | <b>1</b>     | <b>0</b>   | <b>0</b> | <b>0</b>   | <b>0</b>    |
| <b>1</b>    | <b>1</b>     | <b>1</b>   | <b>1</b> | <b>0</b>   | <b>0</b>    |
| <b>0</b>    | <b>0</b>     | <b>1</b>   | <b>0</b> | <b>1</b>   | <b>1</b>    |

# TF-IDF (Term Frequency-Inverse Document Frequency)

- TF-IDF (Term Frequency-Inverse Document Frequency) is a way of measuring how relevant a word is to a document in a collection of documents.
- This is done by multiplying two metrics:
- **Term Frequency (TF)**: how many times a word appears in a document.
- **Inverse Document Frequency (IDF)**: the inverse document frequency of the word across a collection of documents. Rare words have high scores, common words have low scores.

# How to compute TF-IDF

- Suppose we are looking for documents using the query  $Q$  and our database is composed of the documents  $D1$ ,  $D2$ , and  $D3$ .
- $Q$ : The cat.
- $D1$ : The cat is on the mat.
- $D2$ : My dog and cat are the best.
- $D3$ : The locals are playing.

# How to compute TF-IDF

- $\text{TF}(\text{word}, \text{document}) = \text{“number of occurrences of the word in the document”} / \text{“number of words in the document”}$
- Let's compute the TF scores of the words “the” and “cat” (i.e. the query words) with respect to the documents  $D1$ ,  $D2$ , and  $D3$ .
- $\text{TF}(\text{“the”}, D1) = 2/6 = 0.33$
- $\text{TF}(\text{“the”}, D2) = 1/7 = 0.14$
- $\text{TF}(\text{“the”}, D3) = 1/4 = 0.25$

# How to compute TF-IDF

- $\text{TF}(\text{"cat"}, D1) = 1/6 = 0.17$
- $\text{TF}(\text{"cat"}, D2) = 1/7 = 0.14$
- $\text{TF}(\text{"cat"}, D3) = 0/4 = 0$

# How to compute TF-IDF

- IDF can be calculated by taking the total number of documents, dividing it by the number of documents that contain a word, and calculating the logarithm. If the word is very common and appears in many documents, this number will approach 0. Otherwise, it will approach 1.
- $\text{IDF}(\text{word}) = \log(\text{number of documents} / \text{number of documents that contain the word})$



# How to compute TF-IDF

- Let's compute the IDF scores of the words “the” and “cat”.
- $\text{IDF}(\text{“the”}) = \log(3/3) = \log(1) = 0$
- $\text{IDF}(\text{“cat”}) = \log(3/2) = 0.18$

# How to compute TF-IDF

- Multiplying TF and IDF gives the TF-IDF score of a word in a document. The higher the score, the more relevant that word is in that particular document.
- $\text{TF-IDF}(\text{word}, \text{document}) = \text{TF}(\text{word}, \text{document}) * \text{IDF}(\text{word})$

# How to compute TF-IDF

- Let's compute the TF-IDF scores of the words “the” and “cat”.
- $\text{TF-IDF}(\text{“the”}, D1) = 0.33 * 0 = 0$
- $\text{TF-IDF}(\text{“the”}, D2) = 0.14 * 0 = 0$
- $\text{TF-IDF}(\text{“the”}, D3) = 0.25 * 0 = 0$
- $\text{TF-IDF}(\text{“cat”}, D1) = 0.17 * 0.18 = 0.0306$
- $\text{TF-IDF}(\text{“cat”}, D2) = 0.14 * 0.18 = 0.0252$
- $\text{TF-IDF}(\text{“cat”}, D3) = 0 * 0 = 0$

# How to compute TF-IDF

- The next step is to use a ranking function to order the documents according to the TF-IDF scores of their words. We can use the average TF-IDF word scores over each document to get the ranking of  $D1$ ,  $D2$ , and  $D3$  with respect to the query  $Q$ .
- Average TF-IDF of  $D1 = (0 + 0.0306) / 2 = 0.0153$
- Average TF-IDF of  $D2 = (0 + 0.0252) / 2 = 0.0126$
- Average TF-IDF of  $D3 = (0 + 0) / 2 = 0$

# The use of TF-IDF in Machine Learning

- TF-IDF is often used to transform text into a vector of numbers, otherwise known as text vectorization, where the numbers of the vectors are meant to somehow represent the content of the text.
- TF-IDF gives us a way to associate each word in a document with a number that represents how relevant each word is in that document.
- Such numbers can be then used as features of machine learning models.

- Q: The dog
- D1: The dog is playing
- D2: my dog and cat are my pets
- D3: Dogs were the sweetest
- D4: the cat is picking mouse

# Parts of speech tagging

- Part-of-Speech (POS) tagging is a fundamental task in Natural Language Processing (NLP) that involves assigning a grammatical category (such as noun, verb, adjective, etc.) to each word in a sentence.
- The goal is to understand the syntactic structure of a sentence and identify the grammatical roles of individual words.
- POS tagging provides essential information for various NLP applications, including text analysis, machine translation, and information retrieval.

# POS Tags:

- POS tags are short codes representing specific parts of speech. Common POS tags include:
- Noun (NN)
- Verb (VB)
- Adjective (JJ)
- Adverb (RB)
- Pronoun (PRP)
- Preposition (IN)
- Conjunction (CC)
- Determiner (DT)
- Interjection (UH)



# Parts of speech tagging

- One of the core tasks in **Natural Language Processing (NLP)** is **Parts of Speech (PoS) tagging**, which is giving each word in a text a grammatical category, such as nouns, verbs, adjectives, and adverbs.
- Through improved comprehension of phrase structure and semantics, this technique makes it possible for machines to study and comprehend human language more accurately.

# Parts of speech tagging

- In many NLP applications, including machine translation, sentiment analysis, and information retrieval, PoS tagging is essential.
- PoS tagging serves as a link between language and machine understanding, enabling the creation of complex language processing systems and serving as the foundation for advanced linguistic analysis.

# Parts of speech tagging

- Parts of Speech tagging is a linguistic activity in Natural Language Processing (NLP) wherein each word in a document is given a particular part of speech (adverb, adjective, verb, etc.) or grammatical category.
- Through the addition of a layer of syntactic and semantic information to the words, this procedure makes it easier to comprehend the sentence's structure and meaning.

# Parts of speech tagging

- Parts of Speech tagging is a linguistic activity in Natural Language Processing (NLP) wherein each word in a document is given a particular part of speech (adverb, adjective, verb, etc.) or grammatical category.
- Through the addition of a layer of syntactic and semantic information to the words, this procedure makes it easier to comprehend the sentence's structure and meaning.

# Example of POS Tagging

- Consider the sentence: “The quick brown fox jumps over the lazy dog.”
- “The” is tagged as determiner (DT)
- “quick” is tagged as adjective (JJ)
- “brown” is tagged as adjective (JJ)
- “fox” is tagged as noun (NN)
- “jumps” is tagged as verb (VBZ)
- “over” is tagged as preposition (IN)
- “the” is tagged as determiner (DT)
- “lazy” is tagged as adjective (JJ)
- “dog” is tagged as noun (NN)

# Workflow of POS Tagging in NLP

- **Tokenization:** Divide the input text into discrete tokens, which are usually units of words or subwords. The first stage in NLP tasks is tokenization.
- **Loading Language Models:** To utilize a library such as NLTK or SpaCy, be sure to load the relevant language model. These models offer a foundation for comprehending a language's grammatical structure since they have been trained on a vast amount of linguistic data.
- **Text Processing:** If required, preprocess the text to handle special characters, convert it to lowercase, or eliminate superfluous information. Correct PoS labeling is aided by clear text.

# Workflow of POS Tagging in NLP

- **Linguistic Analysis:** To determine the text's grammatical structure, use linguistic analysis. This entails understanding each word's purpose inside the sentence, including whether it is an adjective, verb, noun, or other.
- **Part-of-Speech Tagging:** To determine the text's grammatical structure, use linguistic analysis. This entails understanding each word's purpose inside the sentence, including whether it is an adjective, verb, noun, or other.
- **Results Analysis:** Verify the accuracy and consistency of the PoS tagging findings with the source text. Determine and correct any possible problems or mistagging.

# Named Entity Recognition

- **Named Entity Recognition (NER)** is a technique in **natural language processing (NLP)** that focuses on identifying and classifying entities.
- The purpose of NER is to automatically extract structured information from unstructured text, enabling machines to understand and categorize entities in a meaningful manner for various applications like text summarization, building knowledge graphs, question answering, and knowledge graph construction.



# Named Entity Recognition

- **Name-entity recognition (NER)** is also referred to as **entity identification**, **entity chunking**, and **entity extraction**.
- NER is the component of information extraction that aims to identify and categorize named entities within unstructured text.
- NER involves the identification of key information in the text and classification into a set of predefined categories.
- An entity is the thing that is consistently talked about or refer to in the text, such as person names, organizations, locations, time expressions, quantities, percentages and more predefined categories.

- NER system find applications across various domains, including question answering, information retrieval and machine translation. NER plays an important role in enhancing the precision of other NLP tasks like part-of-speech tagging and parsing. At its core, NLP is just a two-step process, below are the two steps that are involved:
  - Detecting the entities from the text
  - Classifying them into different categories

# Ambiguity in NER

- For a person, the category definition is intuitively quite clear, but for computers, there is some ambiguity in classification. Let's look at some ambiguous examples:
  - England (Organization) won the 2019 world cup vs The 2019 world cup happened in England (Location).
  - Washington (Location) is the capital of the US vs The first president of the US was Washington (Person).

# How Named Entity Recognition (NER) works?

- The NER system analyses the entire input text to identify and locate the named entities.
- The system then identifies the sentence boundaries by considering capitalization rules. It recognizes the end of the sentence when a word starts with a capital letter, assuming it could be the beginning of a new sentence. Knowing sentence boundaries aids in contextualizing entities within the text, allowing the model to understand relationships and meanings.
- NER can be trained to classify entire documents into different types, such as invoices, receipts, or passports. Document classification enhances the versatility of NER, allowing it to adapt its entity recognition based on the specific characteristics and context of different document types.

## How Named Entity Recognition (NER) works?

- NER employs machine learning algorithms, including supervised learning, to analyze labeled datasets. These datasets contain examples of annotated entities, guiding the model in recognizing similar entities in new, unseen data.
- Through multiple training iterations, the model refines its understanding of contextual features, syntactic structures, and entity patterns, continuously improving its accuracy over time.
- The model's ability to adapt to new data allows it to handle variations in language, context, and entity types, making it more robust and effective.

# Lexicon Based Method

- The NER uses a dictionary with a list of words or terms. The process involves checking if any of these words are present in a given text. However, this approach isn't commonly used because it requires constant updating and careful maintenance of the dictionary to stay accurate and effective.

# Rule Based Method

- The Rule Based NER method uses a set of predefined rules guides the extraction of information.
- These rules are based on patterns and context. Pattern-based rules focus on the structure and form of words, looking at their morphological patterns.
- On the other hand, context-based rules consider the surrounding words or the context in which a word appears within the text document.
- This combination of pattern-based and context-based rules enhances the precision of information extraction in Named Entity Recognition (NER).

# Machine Learning-Based Method

- One way is to train the model for multi-class classification using different machine learning algorithms, but it requires a lot of labelling.
- In addition to labelling the model also requires a deep understanding of context to deal with the ambiguity of the sentences.
- This makes it a challenging task for a simple machine learning algorithm.



# Deep Learning Based Method

- Deep learning NER system is much more accurate than previous method, as it is capable to assemble words.
- This is due to the fact that it used a method called word embedding, that is capable of understanding the semantic and syntactic relationship between various words.
- It is also able to learn analyzes topic specific as well as high level words automatically.
- This makes deep learning NER applicable for performing multiple tasks.
- Deep learning can do most of the repetitive work itself, hence researchers for example can use their time more efficiently.

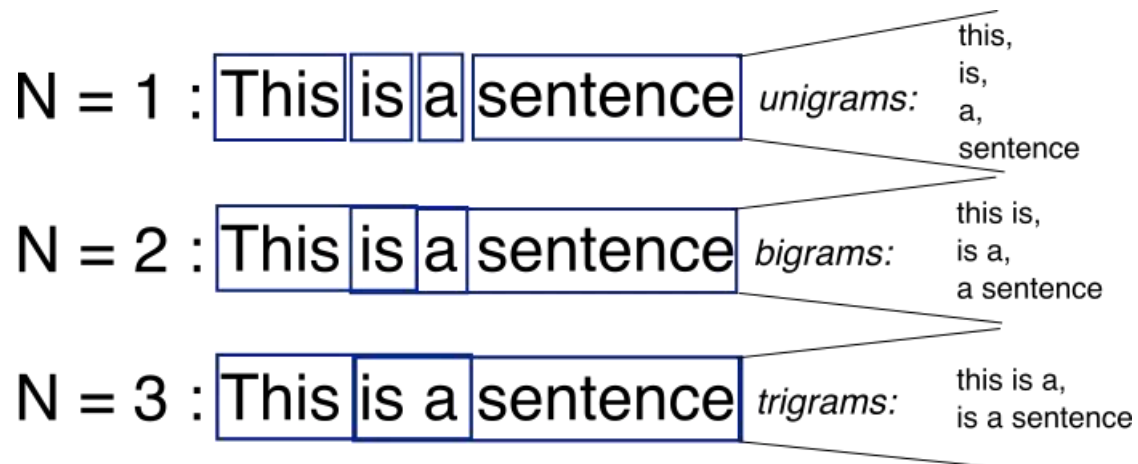
# N Grams

- N-grams are contiguous sequences of 'n' items, typically words in the context of NLP.
- These items can be characters, words, or even syllables, depending on the granularity desired. The value of 'n' determines the order of the N-gram.

# N grams Example

- **Unigrams (1-grams):** Single words, e.g., “cat,” “dog.”
- **Bigrams (2-grams):** Pairs of consecutive words, e.g., “natural language,” “deep learning.”
- **Trigrams (3-grams):** Triplets of consecutive words, e.g., “machine learning model,” “data science approach.”
- 4-grams, 5-grams, etc.: Sequences of four, five, or more consecutive words.

# N grams Example



# Smoothing

- In Natural Language Processing (NLP), **smoothing** refers to techniques used to handle the issue of zero probabilities, particularly in models like **language models**.
- When estimating the likelihood of a sequence of words (e.g., in n-gram models), some word combinations might not appear in the training data.
- This results in zero probability, which can severely affect model performance, especially in tasks like text generation or speech recognition.

# Types of smoothing

- Additive (Laplace) Smoothing
- Good-Turing Smoothing
- Kneser-Ney Smoothing
- Back-off Smoothing
- Interpolation Smoothing
- **Stupid Back-off**

# Additive (Laplace) Smoothing

- **Laplace Smoothing** (also called **Additive Smoothing**) is one of the most widely used methods. In this approach, a small constant (usually 1) is added to the counts of all possible events in the model

- For example, in a bigram language model:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

Where:

- $C(w_{i-1}, w_i)$  is the count of the bigram  $(w_1, w_2)$ ,
- $C(w_{i-1})$  is the count of the unigram  $w_{i-1}$ ,
- $V$  is the size of the vocabulary.

# Good-Turing Smoothing

- This method is based on the idea of adjusting the probability of unseen events by looking at the frequency of events that occur once or twice in the training data.
- Good-Turing smoothing estimates the probability of unseen n-grams based on how many n-grams appear once, twice, and so on.

The formula for adjusting the probability of unseen events is:

$$P(w) = \frac{(c(w) + 1) \times N_c}{N_{c+1}}$$

Where:

- $c(w)$  is the count of word  $w$ ,
- $N_c$  is the number of words that appear  $c$  times,
- $N_{c+1}$  is the number of words that appear  $c + 1$  times.



# Kneser-Ney Smoothing

- A more sophisticated approach often used in n-gram models.
- Kneser-Ney smoothing first adjusts for the frequency of n-grams and then incorporates the diversity of the continuation of the n-gram (i.e., the number of distinct words that follow a given context).
- The idea is that lower-order probabilities (such as unigrams) should be smoothed based on the contexts they appear in, rather than just their raw frequency.

# Back-off Smoothing

- Back-off smoothing involves backing off to a lower-order model (e.g., going from a trigram to a bigram or from bigram to unigram) when higher-order n-grams have not been observed in the training data.
- This technique assigns probability mass to lower-order n-grams when higher-order ones are not available.

# Interpolation Smoothing

- Interpolation combines multiple models (e.g., bigram, trigram) by assigning each model a weight. This is typically used when a language model needs to combine the information from multiple n-gram models to improve prediction accuracy. The probability of a word is a weighted sum of the probabilities from different models.

# Stupid Back-off

- This is a simplified variant of back-off smoothing. Instead of estimating the probability from the lower-order model using a learned interpolation weight, it assigns a fixed back-off factor (often 0.4) when falling back to a lower-order model.